Python 30-Day Workbook

30 days • test-driven development • Python basics to LeetCode foundations

How to Use This

Follow a strict TDD loop: write one failing test, implement the smallest passing solution, then refactor.

Use pytest for every challenge. Keep each day's code in its own folder with solution.py and test_solution.py.

Do not read the expected solution until you have made a real attempt.

LeetCode days are included twice a week to build pattern recognition gradually.

Suggested Folder Structure

```
python-practice/
  day01/
    solution.py
    test_solution.py
  day02/
    solution.py
    test_solution.py
  ...
```

Pytest Quick Start

```
pip install pytest
pytest
```

Each day in this workbook includes the problem statement, example input/output, starter code, TDD steps, and a ready-to-use test file.

Day 1 — Swap Two Variables

Target function: swap_values(a, b)

Problem

Write a function that takes two values and returns them in swapped order.

Example Input and Output

Example	Value
Input	swap_values(3, 5)
Output	(5, 3)

Write These Tests First

```
from solution import swap_values

def test_swap_integers():
    assert swap_values(3, 5) == (5, 3)

def test_swap_strings():
    assert swap_values("a", "b") == ("b", "a")

def test_swap_same_values():
    assert swap_values(7, 7) == (7, 7)
```

TDD Steps

- Write one failing test for integers first.
- Return a hard-coded value just to watch the red-green cycle work.
- Generalize the implementation so any two values are swapped.
- Add tests for strings and identical values.

Starter Code

```
def swap_values(a, b):
    # TODO: return the values in swapped order
    pass
```

What to Pay Attention To

This is a tiny challenge, but it sets up the habit of writing a test before implementation.

Day 2 — Even or Odd

Target function: even_or_odd(n)

Problem

Return 'even' if the input integer is even, otherwise return 'odd'.

Example Input and Output

Example	Value
Input	even_or_odd(10)
Output	'even'

Write These Tests First

```
from solution import even_or_odd

def test_even_number():
    assert even_or_odd(10) == "even"

def test_odd_number():
    assert even_or_odd(7) == "odd"

def test_zero():
    assert even_or_odd(0) == "even"

def test_negative_odd():
    assert even_or_odd(-3) == "odd"
```

TDD Steps

- Start with a single even-number test.
- Write the smallest code to satisfy that test.
- Add an odd-number test, then add 0 and a negative value.
- Refactor only if the code becomes messy.

Starter Code

```
def even_or_odd(n):
    # TODO: return "even" or "odd"
    pass
```

What to Pay Attention To

The `%` operator is central here. `n % 2 == 0` means there is no remainder when dividing by 2.

Day 3 — Sum From 1 to n

Target function: sum_to_n(n)

Problem

Return the sum of all integers from 1 up to n. For this workbook, return 0 for non-positive numbers.

Example Input and Output

Example	Value
Input	sum_to_n(5)
Output	15

Write These Tests First

```
from solution import sum_to_n

def test_sum_to_5():
    assert sum_to_n(5) == 15

def test_sum_to_1():
    assert sum_to_n(1) == 1

def test_sum_to_0():
    assert sum_to_n(0) == 0

def test_negative_returns_zero():
    assert sum_to_n(-3) == 0
```

TDD Steps

- Make `sum_to_n(5)` pass first.
- Add the `n = 1` test next.
- Define behavior for 0 and negative numbers with tests before coding.
- Once the loop version works, you may refactor to a formula.

Starter Code

```
def sum_to_n(n):
    # TODO: return the sum from 1 to n
    pass
```

What to Pay Attention To

A loop is perfectly fine for the first solution. The arithmetic formula is a good refactor once the behavior is covered by tests.

Day 4 — Palindrome Checker

Target function: is_palindrome(text)

Problem

Return True if a string reads the same forward and backward, otherwise False.

Example Input and Output

Example	Value
Input	is_palindrome('level')
Output	True

Write These Tests First

```
from solution import is_palindrome

def test_level_is_palindrome():
    assert is_palindrome("level") is True

def test_hello_is_not_palindrome():
    assert is_palindrome("hello") is False

def test_single_character():
    assert is_palindrome("a") is True

def test_empty_string():
    assert is_palindrome("") is True
```

TDD Steps

- Start with a known palindrome such as 'level'.
- Add a clearly non-palindromic word.
- Add edge cases: one character and empty string.
- After passing tests, consider a stretch version that ignores case and spaces.

Starter Code

```
def is_palindrome(text):
    # TODO: return True if text is a palindrome
    pass
```

What to Pay Attention To

String slicing with `[::-1]` creates a reversed copy of the string.

Day 5 — Largest Number in a List

Target function: find_max(numbers)

Problem

Return the largest number in a list. Raise ValueError for an empty list.

Example Input and Output

Example	Value
Input	find_max([3, 8, 2, 10, 4])
Output	10

Write These Tests First

```
from solution import find_max
import pytest

def test_find_max_normal_list():
    assert find_max([3, 8, 2, 10, 4]) == 10

def test_find_max_single_item():
    assert find_max([7]) == 7

def test_find_max_negative_numbers():
    assert find_max([-5, -2, -9]) == -2

def test_find_max_empty_list():
    with pytest.raises(ValueError):
        find_max([])
```

TDD Steps

- Write the normal-list test first.
- Decide what empty input should do and encode that decision in a test.
- Implement a loop-based version before using `max()` if you want practice.
- Refactor only after all tests are green.

Starter Code

```
def find_max(numbers):
    # TODO: return the largest value
    pass
```

What to Pay Attention To

Writing the manual loop teaches comparison logic. You can later compare it to Python's built-in `max()`.

Day 6 — LeetCode - Two Sum

Target function: two_sum(nums, target)

Problem

Given a list of integers and a target value, return the indices of two numbers whose sum equals the target.

Example Input and Output

Example	Value
Input	two_sum([2, 7, 11, 15], 9)
Output	[0, 1]

Write These Tests First

```
from solution import two_sum

def test_example_case():
    assert two_sum([2, 7, 11, 15], 9) == [0, 1]

def test_second_example():
    assert two_sum([3, 2, 4], 6) == [1, 2]

def test_duplicates():
    assert two_sum([3, 3], 6) == [0, 1]
```

TDD Steps

- Get the first example passing with a brute-force nested-loop solution.
- Add the second example to force generality.
- Add a duplicate-value test.
- Refactor to a dictionary-based one-pass solution once behavior is covered.

Starter Code

```
def two_sum(nums, target):
    # TODO: return the indices of two values that add up to target
    pass
```

What to Pay Attention To

The dictionary stores numbers already seen and their indices. That cuts the time complexity from $O(n^2)$ to $O(n)$.

Day 7 — Remove Duplicates While Preserving Order

Target function: remove_duplicates(items)

Problem

Return a new list with duplicates removed while keeping the first occurrence of each item.

Example Input and Output

Example	Value
Input	remove_duplicates([1, 2, 2, 3, 1, 4])
Output	[1, 2, 3, 4]

Write These Tests First

```
from solution import remove_duplicates

def test_remove_duplicates_normal():
    assert remove_duplicates([1, 2, 2, 3, 1, 4]) == [1, 2, 3, 4]

def test_no_duplicates():
    assert remove_duplicates([1, 2, 3]) == [1, 2, 3]

def test_empty_list():
    assert remove_duplicates([]) == []

def test_all_same():
    assert remove_duplicates([5, 5, 5]) == [5]
```

TDD Steps

- Write the normal-case test first.
- Use a result list and add items only if not already present.
- Add an empty-list test and an all-same-values test.
- Refactor only if readability improves.

Starter Code

```
def remove_duplicates(items):
    # TODO: remove duplicates and preserve order
    pass
```

What to Pay Attention To

A set removes duplicates quickly, but it does not preserve order in the same way. This exercise is specifically about order-preserving behavior.

Day 8 — Maximum of Two Numbers

Target function: max_of_two(a, b)

Problem

Return the larger of two numbers. If the numbers are equal, return that value.

Example Input and Output

Example	Value
Input	max_of_two(10, 5)
Output	10

Write These Tests First

```
from solution import max_of_two

def test_first_is_larger():
    assert max_of_two(10, 5) == 10

def test_second_is_larger():
    assert max_of_two(2, 8) == 8

def test_equal_values():
    assert max_of_two(4, 4) == 4

def test_negative_values():
    assert max_of_two(-1, -5) == -1
```

TDD Steps

- Start with the obvious larger-first case.
- Add a larger-second case to stop hard-coded logic.
- Then add equal values and negatives.
- Keep the function short and readable.

Starter Code

```
def max_of_two(a, b):
    # TODO: return the larger value
    pass
```

What to Pay Attention To

This is a good place to practice writing tiny, precise tests.

Day 9 — Longest Word in a List

Target function: longest_word(words)

Problem

Return the longest word in a list. If there is a tie, return the first longest word. Raise ValueError for an empty list.

Example Input and Output

Example	Value
Input	longest_word(['cat', 'elephant', 'dog'])
Output	'elephant'

Write These Tests First

```
from solution import longest_word
import pytest

def test_longest_word_normal():
    assert longest_word(["cat", "elephant", "dog"]) == "elephant"

def test_tie_returns_first():
    assert longest_word(["bat", "car", "sun"]) == "bat"

def test_single_word():
    assert longest_word(["hello"]) == "hello"

def test_empty_list():
    with pytest.raises(ValueError):
        longest_word([])
```

TDD Steps

- Write the standard example first.
- Add a tie test so the 'first longest' rule is explicit.
- Decide behavior for empty input and encode it with a failing test.
- Refactor to the cleanest correct version after tests pass.

Starter Code

```
def longest_word(words):
    # TODO: return the longest word
    pass
```

What to Pay Attention To

Using a manual loop makes the tie behavior explicit and easy to reason about.

Day 10 — Character Frequency Counter

Target function: char_frequency(text)

Problem

Return a dictionary showing how many times each character appears in the input string.

Example Input and Output

Example	Value
Input	char_frequency('banana')
Output	{'b': 1, 'a': 3, 'n': 2}

Write These Tests First

```
from solution import char_frequency

def test_banana_counts():
    assert char_frequency("banana") == {"b": 1, "a": 3, "n": 2}

def test_empty_string():
    assert char_frequency("") == {}

def test_single_character():
    assert char_frequency("a") == {"a": 1}

def test_spaces_counted():
    assert char_frequency("a a") == {"a": 2, " ": 1}
```

TDD Steps

- Start with 'banana'.
- Add an empty-string test to define the base case.
- Add a spaces-counted test so you are explicit about behavior.
- Later compare your implementation to `collections.Counter`.

Starter Code

```
def char_frequency(text):
    # TODO: count each character
    pass
```

What to Pay Attention To

This is a foundational dictionary exercise. The `.get()` method keeps the code compact and readable.

Day 11 — Common Elements in Two Lists

Target function: common_elements(list1, list2)

Problem

Return a list of unique values that appear in both lists.

Example Input and Output

Example	Value
Input	common_elements([1, 2, 3, 4], [3, 4, 5, 6])
Output	[3, 4]

Write These Tests First

```
from solution import common_elements

def test_common_elements_normal():
    assert common_elements([1, 2, 3, 4], [3, 4, 5, 6]) == [3, 4]

def test_no_common_elements():
    assert common_elements([1, 2], [3, 4]) == []

def test_duplicates_in_input():
    assert common_elements([1, 1, 2, 3], [1, 1, 3]) == [1, 3]

def test_empty_input():
    assert common_elements([], [1, 2]) == []
```

TDD Steps

- Write the normal-case test first.
- Add a duplicates-in-input case to confirm output uniqueness.
- Decide whether output order matters; here, preserve the order from the first list.
- Refactor after behavior is stable.

Starter Code

```
def common_elements(list1, list2):
    # TODO: return unique common elements
    pass
```

What to Pay Attention To

A pure set intersection is shorter, but this version preserves the order of the first list.

Day 12 — Student Record Lookup

Target function: get_student_mark(records, name)

Problem

Given a dictionary mapping student names to marks, return the student's mark if found, otherwise return 'Not found'.

Example Input and Output

Example	Value
Input	get_student_mark({'Tom': 70, 'Sara': 85}, 'Sara')
Output	85

Write These Tests First

```
from solution import get_student_mark

def test_student_exists():
    assert get_student_mark({"Tom": 70, "Sara": 85}, "Sara") == 85

def test_student_missing():
    assert get_student_mark({"Tom": 70}, "Mike") == "Not found"

def test_empty_records():
    assert get_student_mark({}, "Tom") == "Not found"
```

TDD Steps

- Start with a student who exists in the records.
- Add a missing-student test next.
- Finish with empty-record input.
- Keep the implementation as small as possible.

Starter Code

```
def get_student_mark(records, name):
    # TODO: return the mark or 'Not found'
    pass
```

What to Pay Attention To

This is an ideal use case for `dict.get()`.

Day 13 — LeetCode - Valid Anagram

Target function: is_anagram(s, t)

Problem

Return True if the two strings are anagrams of each other, otherwise return False.

Example Input and Output

Example	Value
Input	is_anagram('anagram', 'nagaram')
Output	True

Write These Tests First

```
from solution import is_anagram

def test_valid_anagram():
    assert is_anagram("anagram", "nagaram") is True

def test_not_anagram():
    assert is_anagram("rat", "car") is False

def test_empty_strings():
    assert is_anagram("", "") is True

def test_different_lengths():
    assert is_anagram("a", "ab") is False
```

TDD Steps

- Make the basic anagram example pass first.
- Add a negative example next.
- Add empty strings and different lengths.
- Refactor from sorting to counting if you want the more scalable pattern.

Starter Code

```
def is_anagram(s, t):
    # TODO: return True if s and t are anagrams
    pass
```

What to Pay Attention To

Sorting is the easiest first version. Counting characters is a great dictionary pattern to practice.

Day 14 — Word Frequency in a Sentence

Target function: word_frequency(sentence)

Problem

Return a dictionary with the number of times each word appears in a sentence.

Example Input and Output

Example	Value
Input	word_frequency('the cat and the dog')
Output	{'the': 2, 'cat': 1, 'and': 1, 'dog': 1}

Write These Tests First

```
from solution import word_frequency

def test_sentence_word_count():
    assert word_frequency("the cat and the dog") == {
        "the": 2, "cat": 1, "and": 1, "dog": 1
    }

def test_empty_sentence():
    assert word_frequency("") == {}

def test_single_word():
    assert word_frequency("hello") == {"hello": 1}
```

TDD Steps

- Start with a simple sentence split by spaces.
- Add an empty-string case.
- Keep punctuation handling out of scope unless you add tests for it.
- Refactor only when the behavior is fixed by tests.

Starter Code

```
def word_frequency(sentence):
    # TODO: count each word
    pass
```

What to Pay Attention To

This is the word-based version of Day 10's character-count problem.

Day 15 — Count Even Numbers in a List

Target function: count_evens(numbers)

Problem

Return how many even numbers exist in the input list.

Example Input and Output

Example	Value
Input	count_evens([1, 2, 3, 4, 6])
Output	3

Write These Tests First

```
from solution import count_evens

def test_count_evens_normal():
    assert count_evens([1, 2, 3, 4, 6]) == 3

def test_no_evens():
    assert count_evens([1, 3, 5]) == 0

def test_all_evens():
    assert count_evens([2, 4, 6]) == 3

def test_empty_list():
    assert count_evens([]) == 0
```

TDD Steps

- Write the mixed-list case first.
- Add no-evens and all-evens tests.
- Finish with the empty-list test.
- Refactor into the cleanest form you can explain aloud.

Starter Code

```
def count_evens(numbers):
    # TODO: return the count of even values
    pass
```

What to Pay Attention To

This problem strengthens counting logic and loop control.

Day 16 — Move Zeroes

Target function: move_zeroes(nums)

Problem

Return a list where all zeroes are moved to the end while preserving the order of non-zero elements.

Example Input and Output

Example	Value
Input	move_zeroes([0, 1, 0, 3, 12])
Output	[1, 3, 12, 0, 0]

Write These Tests First

```
from solution import move_zeroes

def test_move_zeroes_example():
    assert move_zeroes([0, 1, 0, 3, 12]) == [1, 3, 12, 0, 0]

def test_no_zeroes():
    assert move_zeroes([1, 2, 3]) == [1, 2, 3]

def test_all_zeroes():
    assert move_zeroes([0, 0, 0]) == [0, 0, 0]

def test_empty():
    assert move_zeroes([]) == []
```

TDD Steps

- Get the sample case passing first.
- Add a no-zeroes case so you don't accidentally reorder data.
- Add all-zeroes and empty-list edge cases.
- Refactor to a two-pass or comprehension solution only after tests are green.

Starter Code

```
def move_zeroes(nums):
    # TODO: move all zeroes to the end
    pass
```

What to Pay Attention To

LeetCode often asks for in-place modification, but for this workbook a returned list keeps the function simpler.

Day 17 — Sort Words by Length

Target function: sort_by_length(words)

Problem

Return the words sorted from shortest to longest. If lengths are equal, keep original order.

Example Input and Output

Example	Value
Input	sort_by_length(['apple', 'kiwi', 'banana'])
Output	['kiwi', 'apple', 'banana']

Write These Tests First

```
from solution import sort_by_length

def test_sort_by_length_normal():
    assert sort_by_length(["apple", "kiwi", "banana"]) == ["kiwi", "apple", "banana"]

def test_empty_list():
    assert sort_by_length([]) == []

def test_same_length_words():
    assert sort_by_length(["bat", "cat", "dog"]) == ["bat", "cat", "dog"]
```

TDD Steps

- Start with mixed-length words.
- Add an empty-list case.
- Add equal-length words to confirm stable ordering.
- Refactor using Python's stable sort.

Starter Code

```
def sort_by_length(words):
    # TODO: sort the list by word length
    pass
```

What to Pay Attention To

Python's sort is stable, so equal-length items keep their original order.

Day 18 — Square Numbers

Target function: square_numbers(numbers)

Problem

Return a new list containing the square of each number in the input list.

Example Input and Output

Example	Value
Input	square_numbers([1, 2, 3, 4])
Output	[1, 4, 9, 16]

Write These Tests First

```
from solution import square_numbers

def test_square_numbers_normal():
    assert square_numbers([1, 2, 3, 4]) == [1, 4, 9, 16]

def test_empty_list():
    assert square_numbers([]) == []

def test_negative_numbers():
    assert square_numbers([-2, -1, 0, 3]) == [4, 1, 0, 9]
```

TDD Steps

- Write the straightforward positive-numbers test first.
- Add an empty-list case.
- Add negative values to confirm squaring behavior.
- Refactor from a loop to a list comprehension if desired.

Starter Code

```
def square_numbers(numbers):
    # TODO: return a list of squared values
    pass
```

What to Pay Attention To

This is a clean introduction to list comprehensions.

Day 19 — Safe Division

Target function: safe_divide(a, b)

Problem

Return the result of dividing a by b. If b is zero, return 'Cannot divide by zero'.

Example Input and Output

Example	Value
Input	safe_divide(10, 2)
Output	5.0

Write These Tests First

```
from solution import safe_divide

def test_normal_division():
    assert safe_divide(10, 2) == 5.0

def test_division_by_zero():
    assert safe_divide(10, 0) == "Cannot divide by zero"

def test_negative_division():
    assert safe_divide(-9, 3) == -3.0
```

TDD Steps

- Start with normal division first.
- Add the division-by-zero case before introducing exception handling.
- Add a negative-number case.
- Keep the exception scope tight.

Starter Code

```
def safe_divide(a, b):
    # TODO: handle division safely
    pass
```

What to Pay Attention To

Catching the specific exception is better than using a blanket `except:`.

Day 20 — LeetCode - Valid Parentheses

Target function: is_valid_parentheses(s)

Problem

Given a string containing only brackets `()[]{}`, return True if every opening bracket is closed in the correct order.

Example Input and Output

Example	Value
Input	is_valid_parentheses('()[]{}')
Output	True

Write These Tests First

```
from solution import is_valid_parentheses

def test_valid_case():
    assert is_valid_parentheses("()[]{}") is True

def test_invalid_case():
    assert is_valid_parentheses("(]") is False

def test_nested_valid_case():
    assert is_valid_parentheses("({[]})") is True

def test_unclosed_opening():
    assert is_valid_parentheses("(") is False
```

TDD Steps

- Start with the simplest valid case.
- Add a mismatched case to force stack logic.
- Add a nested valid case.
- Finish with an unclosed opening bracket case.

Starter Code

```
def is_valid_parentheses(s):
    # TODO: validate bracket ordering
    pass
```

What to Pay Attention To

This is a classic stack problem and a good interview pattern to know.

Day 21 — LeetCode - Best Time to Buy and Sell Stock

Target function: max_profit(prices)

Problem

Given daily stock prices, return the maximum profit from exactly one buy and one sell. If no profit is possible, return 0.

Example Input and Output

Example	Value
Input	max_profit([7, 1, 5, 3, 6, 4])
Output	5

Write These Tests First

```
from solution import max_profit

def test_example_case():
    assert max_profit([7, 1, 5, 3, 6, 4]) == 5

def test_no_profit_possible():
    assert max_profit([7, 6, 4, 3, 1]) == 0

def test_single_day():
    assert max_profit([5]) == 0
```

TDD Steps

- Get the sample case passing first.
- Add a decreasing-prices case to define the zero-profit rule.
- Add the single-day edge case.
- Refactor to a one-pass minimum-price tracker.

Starter Code

```
def max_profit(prices):
    # TODO: return the best profit from one buy and one sell
    pass
```

What to Pay Attention To

This is an excellent one-pass interview problem for practicing state tracking.

Day 22 — Factorial

Target function: factorial(n)

Problem

Return the factorial of a non-negative integer n. Raise ValueError for negative input.

Example Input and Output

Example	Value
Input	factorial(5)
Output	120

Write These Tests First

```
from solution import factorial
import pytest

def test_factorial_5():
    assert factorial(5) == 120

def test_factorial_0():
    assert factorial(0) == 1

def test_factorial_1():
    assert factorial(1) == 1

def test_factorial_negative():
    with pytest.raises(ValueError):
        factorial(-1)
```

TDD Steps

- Write the `factorial(5)` test first.
- Add the base cases 0 and 1.
- Add a negative-input test before coding the guard clause.
- Try an iterative version first, then optionally write a recursive version.

Starter Code

```
def factorial(n):
    # TODO: return n!
    pass
```

What to Pay Attention To

Iterative factorial is easier to debug. Recursive factorial is a useful follow-up exercise.

Day 23 — Linear Search

Target function: linear_search(items, target)

Problem

Return the index of the target if present in the list, otherwise return -1.

Example Input and Output

Example	Value
Input	linear_search([4, 2, 7, 9], 7)
Output	2

Write These Tests First

```
from solution import linear_search

def test_target_found():
    assert linear_search([4, 2, 7, 9], 7) == 2

def test_target_not_found():
    assert linear_search([4, 2, 7, 9], 5) == -1

def test_empty_list():
    assert linear_search([], 1) == -1
```

TDD Steps

- Start with a found-target case.
- Add a not-found case next.
- Add empty input to define the base case.
- Return immediately when the target is found.

Starter Code

```
def linear_search(items, target):
    # TODO: return the index of target or -1
    pass
```

What to Pay Attention To

This is one of the most fundamental algorithmic patterns.

Day 24 — Queue Simulation

Target function: SimpleQueue.enqueue / SimpleQueue.dequeue

Problem

Implement a simple queue class with enqueue and dequeue methods. Dequeue should raise IndexError if the queue is empty.

Example Input and Output

Example	Value
Input	q = SimpleQueue(); q.enqueue(1); q.enqueue(2); q.dequeue()
Output	1

Write These Tests First

```
from solution import SimpleQueue
import pytest

def test_queue_enqueue_dequeue():
    q = SimpleQueue()
    q.enqueue(1)
    q.enqueue(2)
    assert q.dequeue() == 1
    assert q.dequeue() == 2

def test_queue_empty_dequeue():
    q = SimpleQueue()
    with pytest.raises(IndexError):
        q.dequeue()
```

TDD Steps

- Start by testing one enqueue followed by one dequeue.
- Add a second item to confirm first-in-first-out behavior.
- Then add the empty-queue error case.
- Refactor only if the class becomes harder to read.

Starter Code

```
class SimpleQueue:
    def __init__(self):
        # TODO: initialize storage
        pass

    def enqueue(self, item):
        # TODO: add an item to the queue
        pass

    def dequeue(self):
        # TODO: remove and return the next item
        pass
```

What to Pay Attention To

Using a list is fine for learning. Later, `collections.deque` is the better queue structure.

Day 25 — First Non-Repeating Character

Target function: first_unique_char(text)

Problem

Return the first character that appears exactly once in the string. Return None if every character repeats.

Example Input and Output

Example	Value
Input	first_unique_char('swiss')
Output	'w'

Write These Tests First

```
from solution import first_unique_char

def test_first_unique_exists():
    assert first_unique_char("swiss") == "w"

def test_all_repeating():
    assert first_unique_char("aabbcc") is None

def test_empty_string():
    assert first_unique_char("") is None

def test_single_char():
    assert first_unique_char("z") == "z"
```

TDD Steps

- Write the `'swiss'` case first.
- Add an all-repeating case to define the None behavior.
- Add empty string and single-character cases.
- A two-pass dictionary approach is a clean solution.

Starter Code

```
def first_unique_char(text):
    # TODO: return the first unique character or None
    pass
```

What to Pay Attention To

The first pass counts. The second pass preserves original order when checking uniqueness.

Day 26 — FizzBuzz Variant

Target function: fizzbuzz(n)

Problem

Return a list of strings from 1 to n using these rules: multiples of 3 become 'Fizz', multiples of 5 become 'Buzz', multiples of both become 'FizzBuzz'.

Example Input and Output

Example	Value
Input	fizzbuzz(5)
Output	['1', '2', 'Fizz', '4', 'Buzz']

Write These Tests First

```
from solution import fizzbuzz

def test_fizzbuzz_5():
    assert fizzbuzz(5) == ["1", "2", "Fizz", "4", "Buzz"]

def test_fizzbuzz_15_last_value():
    assert fizzbuzz(15)[-1] == "FizzBuzz"

def test_fizzbuzz_1():
    assert fizzbuzz(1) == ["1"]

def test_fizzbuzz_0():
    assert fizzbuzz(0) == []
```

TDD Steps

- Start with a very small input like 5.
- Add a case that reaches 15 so the combined rule is exercised.
- Add `n = 1` and `n = 0` edge cases.
- Keep the conditional order correct: check 15 before 3 or 5.

Starter Code

```
def fizzbuzz(n):
    # TODO: return the FizzBuzz list from 1 to n
    pass
```

What to Pay Attention To

FizzBuzz looks simple, but it is useful practice for branch ordering and test completeness.

Day 27 — LeetCode - Binary Search

Target function: binary_search(nums, target)

Problem

Search for a target in a sorted list and return its index, or -1 if not found.

Example Input and Output

Example	Value
Input	binary_search([-1, 0, 3, 5, 9, 12], 9)
Output	4

Write These Tests First

```
from solution import binary_search

def test_target_found():
    assert binary_search([-1, 0, 3, 5, 9, 12], 9) == 4

def test_target_not_found():
    assert binary_search([-1, 0, 3, 5, 9, 12], 2) == -1

def test_single_item_found():
    assert binary_search([5], 5) == 0

def test_empty_list():
    assert binary_search([], 1) == -1
```

TDD Steps

- Start with a found-target case.
- Add a not-found case.
- Add a single-item list and an empty list.
- Trace `left`, `right`, and `mid` on paper while debugging.

Starter Code

```
def binary_search(nums, target):
    # TODO: search for target in a sorted list
    pass
```

What to Pay Attention To

Binary search requires sorted input. That precondition is part of the problem contract.

Day 28 — Reverse a String

Target function: reverse_string(text)

Problem

Return a reversed version of the input string.

Example Input and Output

Example	Value
Input	reverse_string('python')
Output	'nohtyp'

Write These Tests First

```
from solution import reverse_string

def test_reverse_normal_word():
    assert reverse_string("python") == "nohtyp"

def test_reverse_empty():
    assert reverse_string("") == ""

def test_reverse_single_char():
    assert reverse_string("a") == "a"

def test_reverse_with_spaces():
    assert reverse_string("ab cd") == "dc ba"
```

TDD Steps

- Write the normal-word test first.
- Add empty string and single-character cases.
- Add a string with spaces to prove the function reverses every character.
- Try a loop first, then compare to slicing.

Starter Code

```
def reverse_string(text):
    # TODO: return the reversed text
    pass
```

What to Pay Attention To

A loop-based version helps you understand indexing before using shortcuts like `text[::-1]`.

Day 29 — Intersection of Two Arrays

Target function: intersection(nums1, nums2)

Problem

Return the values common to both arrays, with each value appearing only once in the result.

Example Input and Output

Example	Value
Input	intersection([1, 2, 2, 1], [2, 2])
Output	[2]

Write These Tests First

```
from solution import intersection

def test_intersection_normal():
    assert intersection([1, 2, 2, 1], [2, 2]) == [2]

def test_multiple_common_values():
    assert sorted(intersection([4, 9, 5], [9, 4, 9, 8, 4])) == [4, 9]

def test_no_intersection():
    assert intersection([1, 2], [3, 4]) == []

def test_empty_input():
    assert intersection([], [1, 2]) == []
```

TDD Steps

- Write the one-common-value case first.
- Add a multiple-common-values case.
- Then add no-intersection and empty-input tests.
- Returning a list converted from a set is fine here because order is not the focus.

Starter Code

```
def intersection(nums1, nums2):
    # TODO: return unique common values
    pass
```

What to Pay Attention To

This is a good introduction to set intersection.

Day 30 — LeetCode - Longest Common Prefix

Target function: longest_common_prefix(strs)

Problem

Return the longest starting substring shared by all strings in the list. Return an empty string if there is none.

Example Input and Output

Example	Value
Input	longest_common_prefix(['flower', 'flow', 'flight'])
Output	'fl'

Write These Tests First

```
from solution import longest_common_prefix

def test_common_prefix_exists():
    assert longest_common_prefix(["flower", "flow", "flight"]) == "fl"

def test_no_common_prefix():
    assert longest_common_prefix(["dog", "racecar", "car"]) == ""

def test_single_word():
    assert longest_common_prefix(["alone"]) == "alone"

def test_empty_list():
    assert longest_common_prefix([]) == ""
```

TDD Steps

- Start with the classic example that shares 'fl'.
- Add a no-common-prefix case.
- Add single-word and empty-list cases.
- Shrink the candidate prefix until all words start with it.

Starter Code

```
def longest_common_prefix(strs):
    # TODO: return the longest common prefix
    pass
```

What to Pay Attention To

This is a strong final checkpoint because it combines loops, conditions, and string methods.